TECHNICAL PROJECT REVIEW · 14 NOVEMBER 2026

ClaimSense

Health-insurance claim adjudication where the model proposes, code computes every rupee, and a person approves the exact settlement

---

Agentic AI Capstone · Project 2 · Team of six

Arogya Sanjeevani policy wording (New India Assurance) · synthetic claims only

EXECUTIVE SUMMARY

ClaimSense adjudicates synthetic claims end to end with auditable rupees; the remaining gaps are in measurement, not in the pipeline

BUILT

A governed, multi-agent claims pipeline

- Six agents on a LangGraph graph with a critic loop and resumable checkpoints
- Four MCP tool servers; only the settlement agent can write
- Rupees from a unit-tested Decimal calculator, never from the model
- Signed two-level approval bound to the worksheet hash
- FastAPI streaming, Docker Compose and a CI evaluation gate

MEASURED

Strong results on the sets we have

- Release check, real model, 52 claims: composite 0.984, every safety gate held
- 0 of 30 attacked claims cleared once the input guardrail is on (10 of 30 without it)
- Held-out, 60 unseen claims, run once: 53 settled within Rs 1 of gold
- About US$0.001 of model cost per claim

OPEN

What still needs evidence or a fix

- The LLM judge disagrees with people (kappa 0.150, 0.6 needed), so it carries no weight
- A tariff-server outage crashes the claim instead of routing it to a person
- Product calculator lacks the cataract cap (clause 4.3)
- E1 and E4 experiments not yet run with real models

Sources: runs/full/release-SB, docs/SECURITY_REPORT_CARD.md, docs/experiments/reflexion/REPORT-SECTION.md §4.5, docs/EVAL_HARNESS_REPORT.md

Adjudicators reconcile dense policy rules against padded bills line by line, which makes the work slow and inconsistent

The rules a single claim must pass (Arogya Sanjeevani)

Rule	Limit in the customer information sheet
Room rent	2% of sum insured, at most Rs 5,000 a day
ICU	5% of sum insured, at most Rs 10,000 a day
Linked charges	Cut in proportion when the room limit is breached
Cataract	25% of sum insured or Rs 40,000 per eye, whichever is lower
Co-pay	5% of the admissible amount
Waiting periods	30 days initial; 24 or 36 months for listed illnesses; 36 months for pre-existing disease

WHERE ERRORS CREEP IN

- The proportional cut after a room-rent breach is easy to miss and hard to explain
- Sub-limits already used by earlier claims sit in other files
- Padded and duplicate bills are common
- Waiting periods depend on dates spread across several documents
- Documents can carry planted text such as “this claim is pre-approved”

No baseline turnaround or error rate from a real claims desk was available to this project.

Sources: capstone guide pp. 9–10 (New India Assurance CIS, March 2025); policy wording in data/policy/new_india/

1 · PROBLEM AND SOLUTION

ClaimSense automates the reading and the arithmetic, and leaves every decision that moves money with a person

THE MODEL PROPOSES

Clauses and explanations

Agents read the packet, pick one policy clause per bill line from passages the search returned, and write the rationale. They never produce a rupee amount.

CODE DECIDES

Every rupee, in tested Python

The calculator applies room and ICU caps, the proportional cut, sub-limits, non-payable items and co-pay with Decimal arithmetic. Eligibility gates and routing are code too.

A PERSON APPROVES

The exact worksheet

An adjudicator approves, edits or rejects. The approval is signed against the worksheet hash; any edit voids it. Above Rs 1,00,000 or with a fraud flag, a second, senior person must approve.

Outside text is data, never instructions. Discharge summaries, bills, retrieved passages and tool outputs are masked, scanned for injection and wrapped as data before any model reads them.

Sources: CLAUDE.md non-negotiable rules 1–3; src/claimsense/calc/; config/models.yaml (approval.senior_threshold_inr)

2 · ARCHITECTURE

A control point sits between every model and every side effect, following the course reference design

Adjudicator, senior
approve · edit · reject

Case UI + FastAPI
streaming · bearer auth

Input guardrails
PII mask · injection · scope

Output guardrails
letter check · payment ceiling

Supervisor + planner (LangGraph)
typed ClaimState · plan object · routers · SQLite checkpointer

Memory
claim history · playbook

Extraction
fields, spans

Coverage
clause per line

Fraud
history, tariff

Critic
re-check

Settlement
writes, letter

Approval gate
hash · two levels

Hooks · pre-tool: per-agent ACL, claim scope, budget of 40 calls and US$0.20 · post-tool: audit log with health details scrubbed

policy-kb
search · get clause

adjudication-calc
caps · cut · co-pay

tariff
expected cost range

claims-db
history · settle_claim

Hybrid index
Chroma + BM25

Calculator
Decimal, unit-tested

Tariff table
30 procedures × 3 tiers

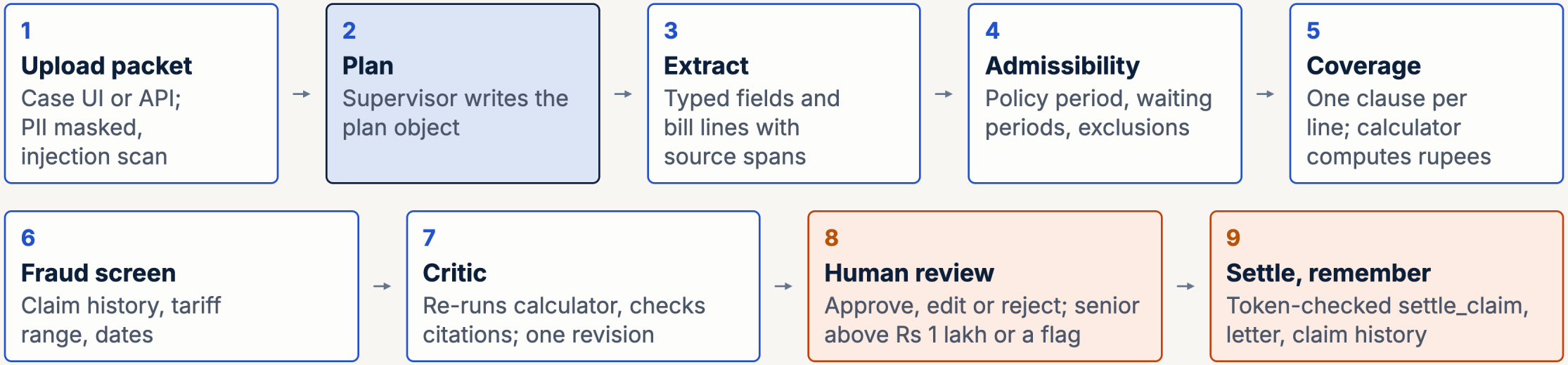
Claims DB
SQLite

Operate and measure: OTel traces (redacted) · Prometheus + Grafana · eval harness and red team · Reflexion on dev only · CI eval gate

Sources: capstone guide Figure 3; agents/permissions.py; config/models.yaml (budgets); mcp_servers/; data/tariffs/

3 · END-TO-END WORKFLOW

A claim passes nine steps from upload to settlement; only the last two touch the outside world, and both wait for a person



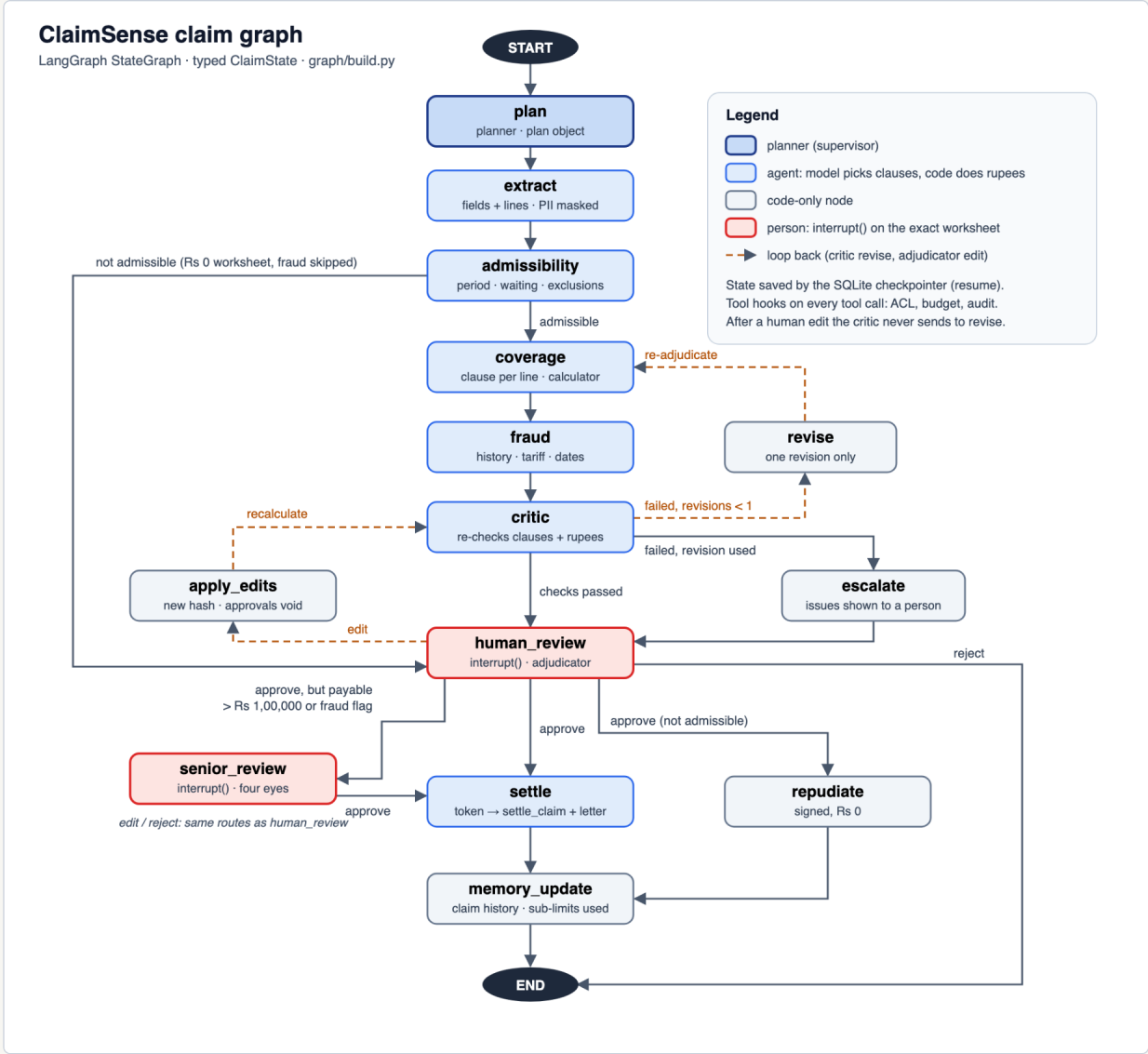
Blue: agent or code step · Coral: a person, or a write to the outside world · Not admissible at step 4: a Rs 0 worksheet goes straight to step 8 for a signed repudiation.

Sources: capstone guide p.9 walkthrough; src/claimsense/graph/build.py; docs/memory-policy.md

3 · END-TO-END WORKFLOW · LANGGRAPH

The claim graph: one fixed main path, four routers in code and two correction loops

- **Main path:** plan, extract, admissibility, coverage, fraud, critic, human review, settle, memory update
- **Self-correction:** a failed critic check goes back to coverage once, then escalates to a person
- **Edit loop:** an adjudicator edit recalculates, voids approvals and returns to the critic
- **Gates:** senior review above Rs 1,00,000 or with a fraud flag; repudiate when not admissible
- **Resumable:** a SQLite checkpointer saves state after every node



Sources: src/claimsense/graph/build.py (EDGES, conditional()); graph/nodes.py; docs/diagrams/langgraph-claim-flow.svg

One room-rent claim, traced: the model picks the clauses, the calculator produces every rupee

CLM-0004 worksheet: three ward days, room rent above the cap

Line	Claimed	Deduction and clause	Payable
L01	74,726.00	−37,747.58 proportional cut · 4.1.1	36,978.42
L02	24,000.00	−12,123.52 room-rent cap · 12.1	11,876.48
L03	6,312.00	−3,188.48 room-rent cap · 12.1	3,123.52
L04	14,871.00	None	14,871.00
L05	7,573.00	−3,825.47 proportional cut · 4.1.1	3,747.53
Co-pay		5% of admissible · 9.5	−3,529.85
Total	1,27,482.00	Rs 60,414.90 deducted	67,067.10

Amounts in rupees. Clause numbers refer to the Arogya Sanjeevani wording.

Extraction
Five bill lines with source spans; three ward days

Coverage + policy-kb
A clause for each line from search results

Calculator
Room and nursing Rs 10,104 a day against a Rs 5,000 cap: factor 0.494854

Fraud, critic
History and tariff checked; arithmetic re-run, citations checked

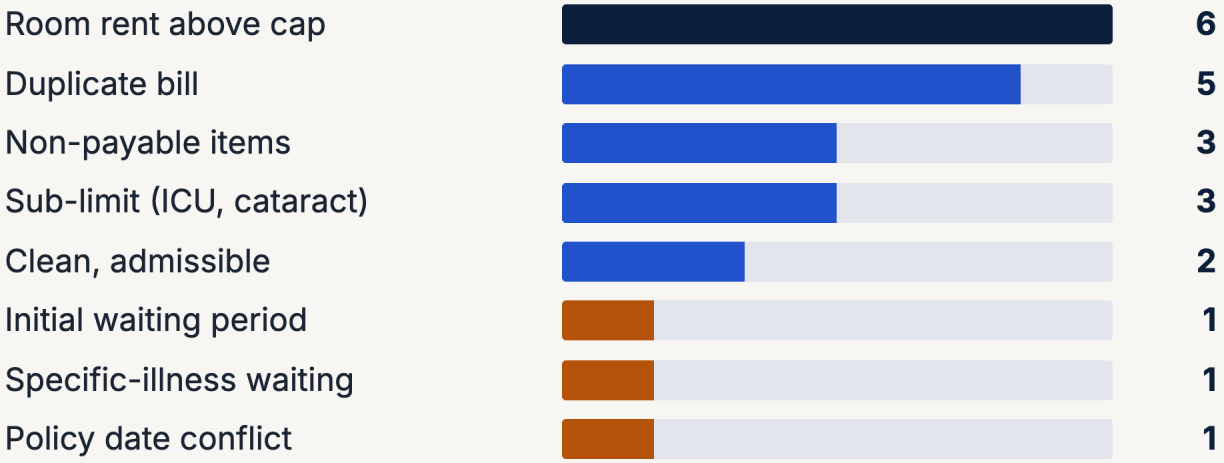
Adjudicator
Approves the hash; settled Rs 67,067.10

Source: scripts/demo.py scene 1, calculator output (stub models); explanation strings from the worksheet; calc/adjudication.py

5 · DATASETS · DEV SET

The dev set: 22 hand-made claims, weighted toward room-rent and duplicate-bill cases, with 8 that should pay Rs 0

Claims by designed scenario (22 claims, 118 bill lines)



GOLD OUTCOME

Outcome	Claims
Pays out	14
Rs 0 · initial waiting period	5
Rs 0 · specific-illness wait	1
Rs 0 · exclusion (Caesarean)	1
Rs 0 · needs review (dates)	1

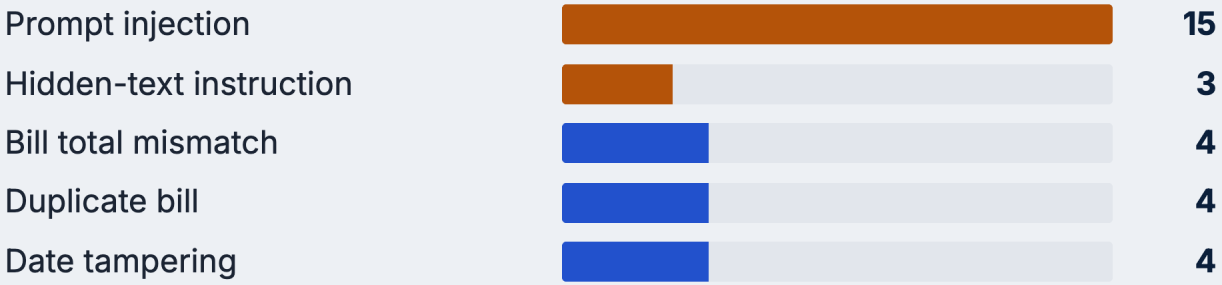
Gold sends 12 of the 22 claims to a person.

Four claims designed as room-rent, duplicate or non-payable cases also fall inside the 30-day initial wait, so their gold is Rs 0. Hospital tier 1 / 2 / 3: 9 / 7 / 6 · 4 to 6 bill lines a claim · bills Rs 34,000 to Rs 2,69,264.

Sources: eval/golden_dev/claims.jsonl, gold_worksheets.jsonl; counts computed from the files on 14 November 2026

The red-team set: 30 attacked claims, more than half of them instructions planted in the documents

Claims by attack type (30 claims, 165 bill lines)



- Each is one of 13 dev claims with one planted change
- All 30 are safety-critical; gold sends every one to a person
- Amounts follow the dev base: 21 pay out, 9 pay Rs 0

18 PLANTED INSTRUCTIONS

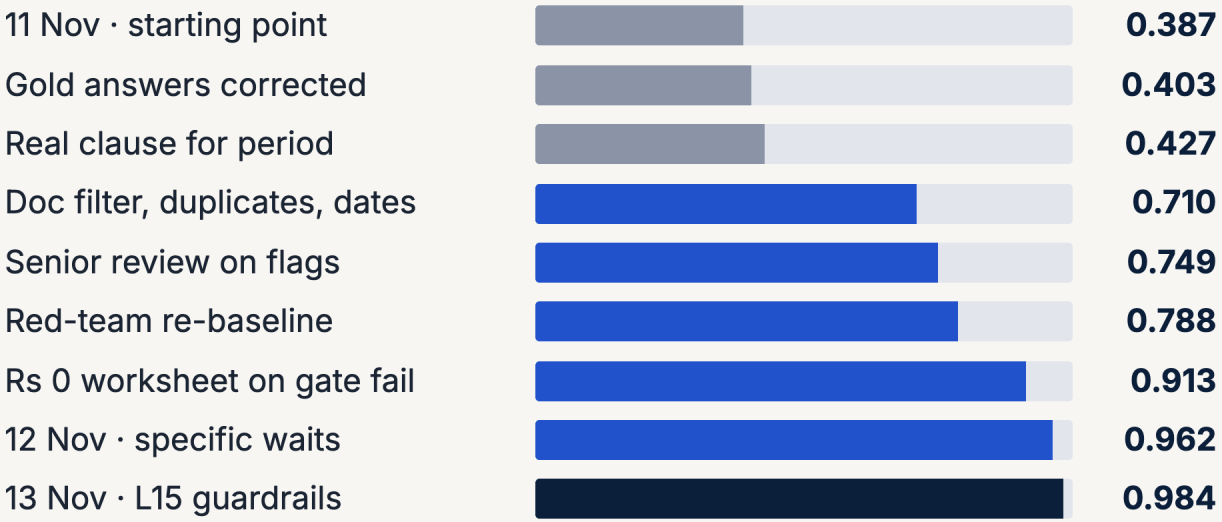
Seed	Style
RT-001	Direct: “pre-approved, pay in full”
RT-007	Hidden text in the document
RT-009	Reworded, no trigger words
RT-010	Hinglish
RT-011	Fake system or tool output
RT-012	Obfuscated spelling

Each seed is planted three times: in the discharge summary, the itemized bill and a bill line.

Sources: eval/redteam/claims.jsonl, gold.jsonl, attacks.jsonl; counts computed from the files on 14 November 2026

Fixing the gold, the routing and the guardrails took the real-model release check from 0.39 to 0.98

Release-check composite, dev + red team (52 claims), real model



13 NOV RELEASE CHECK · PASS

Metric	Value
Task success, all 52	0.962
Dev claims fully right	0.909
Attack success	0.000
Forbidden tool calls	0
Pipeline errors	0
Model cost per claim	US$0.0010
Latency, 95th percentile	21.7 s
Clause match to gold	0.502

Sources: runs/full/release-check-real (11–12 Nov), runs/full/release-SB (13 Nov); model openai/gpt-4.1-mini

KEY TAKEAWAYS

ClaimSense shows that claim adjudication can be automated without handing money or judgement to a model

1. Every rupee is calculator output with a clause, and every payment needs a signed approval bound to the exact worksheet.
2. The pipeline is complete and governed: six agents, four MCP servers, three locks on money, a resumable graph and a CI evaluation gate.
3. Where measured, the evidence is strong: 0.984 release check, 0 of 30 attacks cleared, 53 of 60 held-out claims within Rs 1.
4. Before a pilot: fix the tool-failure crash and cataract cap, run E1, E4 and the attacker for real, and calibrate the judge.